GreeceClean

Complete Project Handover Documentation

Prepared by: Technical Lead / Project Manager

Date: May 2026

Version: 1.0 — Final

Classification: Confidential — Client Handover

Contents: Technical Documentation · Deployment Checklist · Security Audit · Project Retrospective · Pre-Kickoff Stakeholder Questions · Initial Prompt Template

Table of Contents

Technical Documentation

1. Project Overview

GreeceClean is a citizen-facing web application that allows anyone to photograph and report environmental violations — illegal dumping, roadside litter, abandoned vehicles, vandalism — and automatically routes those reports to the responsible Greek municipality for action.

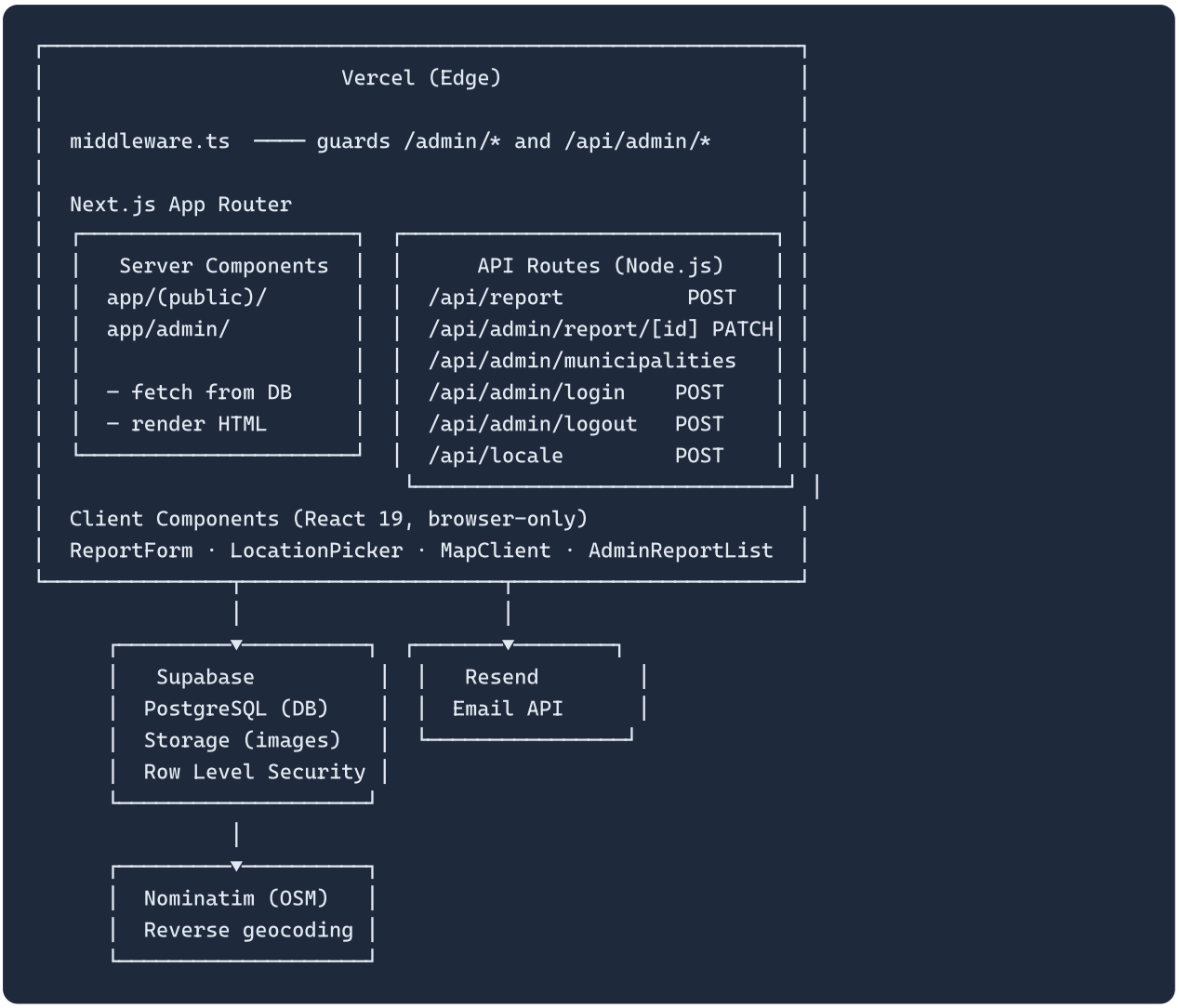
Core user journey:

```
Citizen photographs a problem
  → Uploads photo + confirms location on map
  → Receives a personal tracking URL (/r/<token>)
  → Admin reviews the report in the dashboard
  → Admin forwards to the municipality (automated email)
  → Municipality acts; admin marks resolved
  → Citizen sees progress on their tracking page
```

The platform has three audiences:

Audience	Entry point	Auth
Citizens	`/`, `/report`, `/map`, `/r/[token]`	None — fully public
Administrators	`/admin/dashboard`, `/admin/municipalities`	Password cookie
Municipalities	Receive email notifications only	Not a system user

2. Architecture



Key architectural decisions:

- **No client-side DB mutations.** All writes go through Next.js API routes using the `supabaseAdmin` client (service role). The browser-facing `supabase` client (anon key) is used only for public reads.
- **Server Components for all pages.** Pages fetch data server-side at request time (`force-dynamic`). No client-side data fetching on page load.
- **Middleware at the edge.** `middleware.ts` validates the admin session cookie before Next.js even renders the route.
- **Stub mode.** The entire app runs without Supabase using in-memory seed data, enabling local development with zero external dependencies.

---

3. Tech Stack

Layer	Technology	Version	Purpose
Framework	Next.js	^16.2	App Router, API routes, middleware
UI	React	^19.0	Component model
Language	TypeScript	^5	Strict mode enabled
Styling	Tailwind CSS	^3.4	Utility-first CSS
Database	Supabase (PostgreSQL)	^2.47 JS client	Reports, municipalities, email logs
Storage	Supabase Storage	—	WebP report images
Email	Resend	^6.12	Municipality notification emails
Maps	Leaflet	^1.9	Interactive map picker + public map
Image processing	sharp	^0.33	Server-side WebP compression
EXIF parsing	exifr	^7.1	GPS coordinate extraction from photos
Geocoding	Nominatim (OpenStreetMap)	—	Reverse geocode lat/lng → municipality name

4. Local Development Setup

Prerequisites

- Node.js 20+
- npm 10+
- (Optional) A Supabase project — the app runs in stub mode without one

Steps

```
git clone <repo-url>
cd GreeceClean
npm install
cp .env.local.example .env.local
# Edit .env.local — see Environment Variables section
npm run dev
# → http://localhost:3000
```

Minimum .env.local for full Supabase mode

```
NEXT_PUBLIC_SUPABASE_URL=https://your-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
NEXT_PUBLIC_APP_URL=http://localhost:3000
ADMIN_PASSWORD=any-password-for-dev
ADMIN_COOKIE_SECRET=any-secret-for-dev-min-32-chars
```

`RESEND_API_KEY` and `EMAIL_FROM` are only needed to test email forwarding locally. Omit all `SUPABASE_*` variables to enter stub mode (17 sample reports, no DB required).

---

5. Project Structure

```
GreeceClean/
├── app/
│   ├── layout.tsx                # Root layout: Inter font, LocaleProvider, Header
│   ├── globals.css               # Tailwind base + .btn-primary, .btn-action, .card
│   ├── (public)/
│   │   ├── page.tsx              # Landing page — hero, stats, leaderboard
│   │   ├── report/page.tsx       # Report submission wrapper
│   │   ├── map/page.tsx          # Public Leaflet map
│   │   └── r/[token]/page.tsx    # Citizen tracking page
│   ├── admin/
│   │   ├── login/page.tsx        # Password login form
│   │   ├── dashboard/page.tsx    # Report review (pending / approved / rejected)
│   │   └── municipalities/page.tsx
│   └── api/
│       ├── report/route.ts       # POST — public report submission
│       ├── locale/route.ts       # POST — language preference cookie
│       └── admin/
│           ├── login/route.ts
│           ├── logout/route.ts
│           ├── report/[id]/route.ts
│           └── municipalities/[id]/route.ts
├── components/
│   ├── Header.tsx
│   ├── ReportForm.tsx            # Multi-step report form (client)
│   ├── LocationPicker.tsx        # Leaflet map picker (client, SSR-disabled)
│   ├── MapClient.tsx             # Public read-only map (client, SSR-disabled)
│   ├── MapWrapper.tsx            # dynamic() SSR wrapper
│   ├── LocaleProvider.tsx        # React context: locale + Dictionary
│   ├── LanguageSwitcher.tsx
│   ├── AdminReportList.tsx
│   ├── MunicipalityEmailList.tsx
│   └── CopyButton.tsx
├── lib/
│   ├── supabase.ts               # Two clients: anon (RLS) + admin (service role)
│   ├── geocoding.ts              # reverseGeocode() via Nominatim
│   ├── email.ts                  # sendEmail() wrapper around Resend SDK
│   ├── emailTemplates.ts         # buildMunicipalityReportEmail()
│   ├── seed-data.ts              # 17 sample reports for stub mode
│   └── i18n/
│       ├── types.ts              # Locale type + full Dictionary type
│       ├── index.ts              # getDictionary() + getLocale()
│       ├── el.ts                 # Greek translations
│       ├── en.ts                 # English translations
│       └── de.ts                 # German translations
├── supabase/
│   ├── schema.sql                # Tables, RLS, indexes, triggers
│   ├── email_notifications.sql   # email_logs + UNIQUE constraint
│   └── seed_municipalities.sql   # Real Greek municipality list
├── middleware.ts                 # Edge middleware — guards all /admin/* routes
├── next.config.ts
├── tailwind.config.ts
└── .env.local.example
```

6. Database Schema

`municipalities`

Column	Type	Nullable	Description
`id`	`uuid`	NO	Primary key
`name_el`	`text`	NO	Greek name — UNIQUE
`name_en`	`text`	NO	English name
`email_official`	`text`	YES	Destination for forwarded report emails
`region`	`text`	YES	Greek regional unit name
`created_at`	`timestamptz`	NO	Auto-set on insert

`reports`

Column	Type	Nullable	Description
`id`	`uuid`	NO	Internal primary key
`public_token`	`text`	NO	UNIQUE — 12-char hex for `/r/<token>`
`image_url`	`text`	YES	Supabase Storage public URL
`lat`	`double precision`	NO	Latitude (34.8–41.8)
`lng`	`double precision`	NO	Longitude (19.3–29.7)
`category`	`text`	NO	One of 5 valid values
`status`	`report_status`	NO	Lifecycle state
`is_approved`	`boolean`	NO	Controls public visibility
`municipality_id`	`uuid`	YES	FK → municipalities.id
`description`	`text`	YES	Optional note, max 500 chars
`created_at`	`timestamptz`	NO	Submission timestamp
`updated_at`	`timestamptz`	NO	Auto-updated by trigger

`report_status` **enum:** `pending` , `in_review` , `forwarded` , `resolved` , `rejected`

`email_logs`

Column	Type	Nullable	Description
`id`	`uuid`	NO	Primary key
`report_id`	`uuid`	NO	FK → reports.id ON DELETE CASCADE
`municipality_id`	`uuid`	YES	FK → municipalities.id
`recipient_email`	`text`	NO	Snapshot of address at send time
`status`	`text`	NO	`'sent'` or `'failed'`
`error_message`	`text`	YES	Resend error if failed
`sent_at`	`timestamptz`	NO	Auto-set on insert

RLS Policies

Table	Role	Operation	Rule
`reports`	anon	SELECT	`WHERE is_approved = true`
`reports`	anon	INSERT	`WITH CHECK (true)`
`reports`	anon	UPDATE	**Not allowed**
`reports`	anon	DELETE	**Not allowed**
`municipalities`	anon	SELECT	`USING (true)`
`municipalities`	anon	INSERT/UPDATE/DELETE	**Not allowed**
`email_logs`	anon	ALL	**Not allowed**
service_role	ALL	ALL	Bypasses RLS

Report Categories

Value	Greek label
`illegal_dump`	Παράνομη Χωματερή
`roadside_litter`	Σκουπίδια στο Δρόμο
`abandoned_vehicle`	Εγκαταλελειμμένο Όχημα
`vandalism`	Βανδαλισμός
`other`	Άλλο

7. Environment Variables

Variable	Client?	Required	Description
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Yes	Supabase project URL. Safe to expose — no auth capability.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Yes	Supabase anonymous key. Safe to expose — RLS enforces access.
`SUPABASE_SERVICE_ROLE_KEY`	**No**	Yes	Bypasses RLS. **Never** add `NEXT_PUBLIC_` prefix.
`NEXT_PUBLIC_APP_URL`	Yes	Yes	Full origin (e.g. `https://greececlean.gr`). Used in tracking URLs stored in DB and emails. Set before first real report.
`ADMIN_PASSWORD`	**No**	Yes	Single admin credential. Min 20 random characters.
`ADMIN_COOKIE_SECRET`	**No**	Yes	HMAC signing key. Min 32 random characters. Different from `ADMIN_PASSWORD`.
`RESEND_API_KEY`	**No**	For email	`re_...` format. Only required for municipality forwarding.
`EMAIL_FROM`	**No**	For email	`GreeceClean <noreply@greececlean.gr>`. Domain must match Resend verified domain.

Generate strong secrets:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Run twice — one value for `ADMIN_PASSWORD`, a different one for `ADMIN_COOKIE_SECRET`.

---

8. Authentication

Single shared password — no user accounts in the database.

Login flow

```
POST /api/admin/login  { password: "..." }
  ↓ Compare against ADMIN_PASSWORD
  ↓ match → HMAC-SHA256(ADMIN_PASSWORD, ADMIN_COOKIE_SECRET) → token
  Set-Cookie: admin_session=<token>; HttpOnly; SameSite=Lax; Max-Age=28800
  Redirect → /admin/dashboard
```

Session validation (middleware.ts)

Every request matching `/admin/:path*` or `/api/admin/:path*` (except login/logout routes):

```
const expected = createHmac('sha256', ADMIN_COOKIE_SECRET)
                   .update(ADMIN_PASSWORD).digest('hex')
return cookieValue === expected
```

Session properties: HttpOnly, SameSite=Lax, 8-hour MaxAge, cookie name `admin_session`.

To invalidate all sessions: Rotate `ADMIN_COOKIE_SECRET` in Vercel and redeploy. All existing cookies become invalid.

---

9. API Reference

`POST /api/report` — Public report submission

Request: `multipart/form-data`

Field	Type	Required	Validation
`image`	File	Yes	Max 10 MB
`lat`	string (float)	Yes	34.8–41.8
`lng`	string (float)	Yes	19.3–29.7
`category`	string	Yes	One of 5 valid values
`description`	string	No	Truncated to 500 chars server-side
`hp_field`	string	Yes	Honeypot — must be empty

Processing: FormData parse → honeypot check → field validation → WebP compression → geocoding → storage upload → DB insert

Success response:

```
{ "token": "a1b2c3d4e5f6", "trackingUrl": "https://greececlean.gr/r/a1b2c3d4e5f6" }
```

Status	Condition
400	Invalid request body / missing fields
413	Image > 10 MB
422	Coordinates outside Greece / invalid category / incompressible image
500	Storage or database error

`PATCH /api/admin/report/[id]` — Admin report actions

`action`	DB effect
`approve`	`is_approved=true, status='in_review'`
`reject`	`is_approved=false, status='rejected'`
`mark_cleaned`	`status='resolved'`
`deactivate`	`is_approved=false, status='pending'`
`forward`	`status='forwarded'` + sends email → logs to `email_logs`
`edit`	Updates category, status, municipality_id, description

Forward 207 response (status updated, email failed):

```
{ "ok": true, "warning": "status changed but email failed: <reason>" }
```

`DELETE /api/admin/report/[id]`

Deletes the DB row and removes `<public_token>.webp` from Supabase Storage.

`PATCH /api/admin/municipalities/[id]`

Updates `email_official` (validated email format), `region`, `name_en`.

`POST /api/locale`

Sets `locale` cookie to `el`, `en`, or `de`. Not HttpOnly. 1-year expiry.

---

10. Component Architecture

Server vs Client boundary

Type	Components
Server (async, DB access)	All page files in `app/`
Client (browser APIs, interactivity)	ReportForm, LocationPicker, MapClient, AdminReportList, MunicipalityEmailList, Header, LanguageSwitcher, CopyButton

Leaflet and SSR

Leaflet directly accesses `window` / `document` on import — this crashes SSR. Rule: **every Leaflet component must be loaded with** `dynamic(() ⇒ import( … ), { ssr: false })`.

`MapWrapper.tsx` wraps `MapClient`. `LocationPicker` is loaded via dynamic import inside `ReportForm`.

Context

`LocaleProvider` wraps the entire tree. Any client component can call:

```
const { locale, t } = useLocale()
```

This provides the active locale string and the full translated Dictionary object.

---

11. Internationalisation (i18n)

Three languages: `el` (default), `en`, `de`

Locale resolution: `getLocale()` reads the `locale` cookie server-side on every request. Falls back to `'el'`. Passed to `LocaleProvider` in `app/layout.tsx`.

Changing language: User clicks a flag → `POST /api/locale` sets cookie → `router.refresh()` re-renders server components with new locale.

Admin area: Intentionally Greek-only. All admin strings are hardcoded in Greek — the admin is an internal tool.

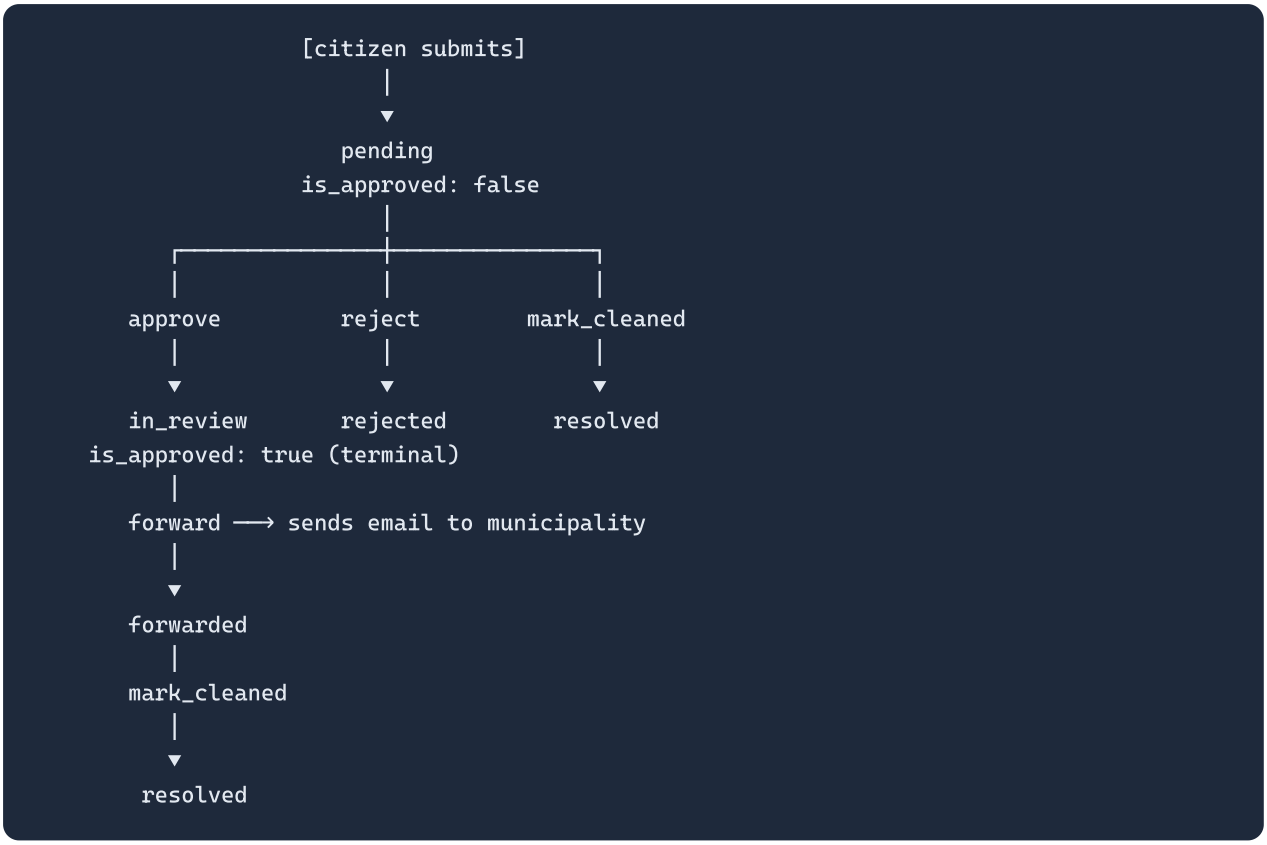
Email templates: Always sent in Greek, regardless of locale.

Adding a new language

1. Create `lib/i18n/xx.ts` implementing the full `Dictionary` type
2. Add `'xx'` to `LOCALES` in `lib/i18n/types.ts`
3. Import and add `xx` to `dicts` in `lib/i18n/index.ts` and `components/LocaleProvider.tsx`
4. Add the flag to `LANGS` in `components/LanguageSwitcher.tsx`

---

12. Report Lifecycle



Visibility: A report appears on the public map only when `is_approved = true`. The tracking page is always accessible by token.

Progress stepper on tracking page:

Step	Done when status is...
Submitted	Always
Verified	`in_review`, `forwarded`, `resolved`
Municipality Notified	`forwarded`, `resolved`
Cleaned Up	`resolved`

13. Image Processing Pipeline

All processing happens server-side in `/api/report/route.ts` using **sharp**.

```
Upload (any format, max 10 MB)
  ↓
Attempt 1: max width 1920px, WebP quality 80
  ↓ if > 500 KB
Attempt 2: max width 1200px, WebP quality 65
  ↓ if > 500 KB
Attempt 3: max width 900px, WebP quality 50
  ↓ if still > 500 KB → 422 error
  ↓
Upload to Supabase Storage as <token>.webp (public)
```

`sharp` is declared as `serverExternalPackages: ['sharp']` in `next.config.ts`. It cannot run on Edge Runtime.

---

14. Location & Geocoding Pipeline

EXIF GPS (client): `exifr.gps(file)` extracts coordinates from the photo. If found and within the Greek bounding box (lat 34.8–41.8, lng 19.3–29.7), they pre-fill the map. If outside Greece, an amber warning is shown. EXIF failure is non-fatal.

Map picker (client): Four ways to set location — click map, drag marker, GPS button (`navigator.geolocation`), Nominatim search (debounced 500ms, restricted to Greece via `countrycodes=gr`).

Reverse geocoding (server): After submission, Nominatim is called at zoom=10 with Greek language preference. Address field priority: `municipality → city → town → village → hamlet → suburb → county → state` . Falls back to zoom=8 if nothing found.

Municipality resolution: Name looked up in DB (exact match, then partial). If unknown, a new row is auto-created so the admin always sees the real location.

---

15. Email Notification System

Forward flow

```
Admin clicks "Forward"
  → UPDATE reports SET status='forwarded'
  → buildMunicipalityReportEmail(report, municipality)
  → sendEmail({ to, subject, html })
  → INSERT email_logs { status: 'sent' | 'failed' }
  → Return 200 (success) or 207 (status updated, email failed)
```

The status transitions to `forwarded` even if the email fails. Failure is logged and surfaced to the admin as a warning — it never blocks the status update.

Email template

Responsive HTML table layout (Outlook-compatible inline CSS), always in Greek. Contains: category, municipality, date, description, coordinates (Google Maps link), report photo, CTA button, tracking URL.

Finding failed notifications

```sql
SELECT r.public_token, m.name_el, el.recipient_email,
       el.error_message, el.sent_at
FROM email_logs el
JOIN reports r ON r.id = el.report_id
LEFT JOIN municipalities m ON m.id = el.municipality_id
WHERE el.status = 'failed'
ORDER BY el.sent_at DESC;
```

---

16. Admin Workflow

Dashboard (`/admin/dashboard`)

Section	Query	Actions
Pending	`is_approved=false AND status ≠ 'rejected'`	Approve, Reject, Mark Cleaned
Approved	`is_approved=true` (max 100)	Forward, Deactivate, Edit, Delete
Rejected	`status='rejected'` (max 50)	Edit, Delete

Municipality Management (`/admin/municipalities`)

All municipalities sorted by `name_el`. Inline edit of `email_official` and `region`. Green dot = has confirmed email. Reports cannot be forwarded to municipalities without a confirmed email.

---

17. Stub Mode (Demo without Supabase)

When `NEXT_PUBLIC_SUPABASE_URL` or `NEXT_PUBLIC_SUPABASE_ANON_KEY` are absent (`isSupabaseConfigured = false`):

Feature	Behaviour
Landing page stats	Counts from `lib/seed-data.ts` (17 items)
Public map	Renders 17 seed report markers
Tracking page	Resolves against seed tokens
Report submission	Returns fake token + `_stub: true`. No DB write.
Admin routes	Return `503 Supabase not configured`

---

18. Styling System

Brand colours:

Token	Hex	Usage
`primary`	`#005BAE`	Deep blue — buttons, links, headings, header
`action`	`#6B8E23`	Olive green — CTAs, success states, progress

Global utility classes (in `globals.css`):

```
.btn-primary   /* blue button — bg-primary text-white rounded-2xl */
.btn-action    /* green button — bg-action text-white rounded-2xl */
.card          /* white card — bg-white rounded-2xl shadow-sm border p-6 */
```

All interactive elements use `rounded-2xl` (1rem) by default.

---

19. Known Limitations & Future Work

Area	Limitation
Rate limiting	No server-side rate limiting on `/api/report`. Honeypot is the only bot mitigation.
Admin auth	Single shared password. No per-user accounts or audit trail.
Nominatim	No explicit rate limiting (1 req/sec policy). Bursts may result in reports saved without municipality.
Pagination	Admin dashboard loads max 100 approved / 50 rejected reports.
`proxy.ts`	Legacy artefact — superseded by `middleware.ts`. Safe to delete.
`lib/notifications.ts`	Legacy stub — superseded by `lib/emailTemplates.ts`. Safe to delete.
`MunicipalityEmailList`	Has the same React Fragment key issue as `AdminReportList` had (not yet fixed).

Suggested future improvements: Turnstile CAPTCHA · Supabase Auth for named admin users · Municipality portal (direct login) · Push notifications for citizens · Bulk forwarding · Report clustering on map.

---

20. How to Extend the Project

Add a new report category

1. Add to the whitelist array in `/api/report/route.ts` and `VALID_CATEGORIES` in `/api/admin/report/[id]/route.ts`
2. Add to `categories` array in `lib/i18n/el.ts`, `en.ts`, `de.ts`
3. Add to `CATEGORIES` in `components/AdminReportList.tsx`
4. Add to `CATEGORY_LABELS` in `lib/emailTemplates.ts`

Add a new language

See Section 11 — Adding a new language.

Add a new admin action

1. Add a new `action` branch in `PATCH /api/admin/report/[id]`

2. Add the button in `AdminReportList.tsx` (mode-conditional, follows existing pattern)
3. `runAction()` handles loading state and error feedback automatically

Add a new database column to reports

1. `ALTER TABLE reports ADD COLUMN IF NOT EXISTS ...` in `schema.sql`
2. Update `REPORT_SELECT` in `app/admin/dashboard/page.tsx`
3. Update the `Report` type in `app/(public)/r/[token]/page.tsx`
4. Update `AdminReport` type in `components/AdminReportList.tsx`
5. Add i18n strings to all three dictionaries if user-facing

Production Deployment Checklist

1. Supabase — Project Setup

Create the project

- [ ] New Supabase project, region `eu-central-1` (Frankfurt) — lowest latency from Greece
- [ ] Enable `pgcrypto` extension (*Database → Extensions*)
- [ ] Note: **Project URL**, **anon key**, **service_role key** from *Settings → API*

Run migrations in order (SQL Editor — each in a separate run)

```
1. supabase/schema.sql
2. supabase/email_notifications.sql
3. supabase/seed_municipalities.sql
```

> ***Important:*** *`schema.sql` ends with 4 sample INSERT rows for Athens/Thessaloniki/Heraklion/Patras. Delete those 4 lines before running, or they will conflict with `seed_municipalities.sql`.*

Verify RLS (*Table Editor → reports → RLS*)

- [ ] `Public can read approved reports` — `USING (is_approved = true)` present
- [ ] `Anyone can submit a report` — INSERT with `WITH CHECK (true)` present
- [ ] No UPDATE or DELETE policy exists for the `anon` role
- [ ] `municipalities` — read-only public policy, no write policies for anon

Create storage bucket

- [ ] *Storage → New Bucket* → name: `reports`, **Public bucket: ON**
- [ ] Confirm URL pattern matches `next.config.ts`: `*.supabase.co/storage/v1/object/public/**`
- [ ] Set file size limit: **10 MB**
- [ ] Add MIME-type restriction: `image/webp` only

---

2. Resend — Email Setup

- [ ] Create Resend account, add sending domain (e.g. `greececlean.gr`)
- [ ] Add DNS records provided by Resend: **SPF, DKIM, DMARC**

- [ ] Wait for domain **Verified** status before sending
- [ ] Generate API key with **Sending access only**
- [ ] Confirm `EMAIL_FROM` domain matches the verified domain

---

3. Vercel — Project Setup

Import & build

- [ ] *New Project → Import Git Repository*
- [ ] Framework: **Next.js** (auto-detected)
- [ ] Node.js version: **20.x** — required for `sharp` native binaries
- [ ] Do **not** enable Edge Runtime — `sharp` requires Node.js

Environment variables (*Settings → Environment Variables*)

Variable	Source	Note
`NEXT_PUBLIC_SUPABASE_URL`	Supabase Settings → API	Safe for browser
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase Settings → API	Safe for browser
`SUPABASE_SERVICE_ROLE_KEY`	Supabase Settings → API	Server-only — never `NEXT_PUBLIC_`
`NEXT_PUBLIC_APP_URL`	Your domain	`https://greececlean.gr` — set before first real report
`ADMIN_PASSWORD`	Generate	Min 20 random chars
`ADMIN_COOKIE_SECRET`	Generate	Min 32 random chars, different from above
`RESEND_API_KEY`	Resend dashboard	`re_...`
`EMAIL_FROM`	Your domain	`GreeceClean <noreply@greececlean.gr>`

Custom domain

- [ ] *Settings → Domains* → add `greececlean.gr` and `www.greececlean.gr`
- [ ] Update `NEXT_PUBLIC_APP_URL` to the final domain **before** first real report is submitted
- [ ] Verify HTTPS is active

4. Security Headers (add to `next.config.ts`)

```
async headers() {
  return [
    {
      source: '/(.*)',
      headers: [
        { key: 'X-Frame-Options',        value: 'DENY' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'Referrer-Policy',        value: 'strict-origin-when-cross-origin' },
        { key: 'Permissions-Policy',     value: 'camera=(), microphone=(),
geolocation=(self)' },
      ],
    },
  ]
},
```

> *`geolocation=(self)` is required — `LocationPicker` calls `navigator.geolocation`.*

5. Pre-Launch Verification (run on staging first)

Report submission flow

- [ ] Upload photo with GPS EXIF → green "Location found" banner appears
- [ ] Upload photo without GPS → amber warning appears
- [ ] Select location, submit → tracking URL returned
- [ ] Open `/r/<token>` → photo, OSM embed, category, municipality, stepper all render
- [ ] Copy button shows "📋 Copy Link" / "✓ Copied!" (not "Tracking link")
- [ ] WhatsApp share link opens correctly

Honeypot test

```
curl -X POST https://your-preview.vercel.app/api/report \
  -F "image=@test.jpg" -F "lat=37.97" -F "lng=23.73" \
  -F "category=other" -F "hp_field=bot-filled-this"
```

Expected: `200` with fake token. Confirm no row in Supabase.

Admin flow

- [ ] `/admin/login` with correct password → dashboard
- [ ] `/admin/login` with wrong password → error
- [ ] Navigate to `/admin/dashboard` in private window without session → redirects to login
- [ ] `GET /api/admin/report/some-id` without session → 307 to login
- [ ] Approve → appears in Approved section
- [ ] Forward → check Resend dashboard for delivery
- [ ] Delete → image removed from Supabase Storage

i18n

- [ ] Switch to English → all UI strings update
- [ ] Switch to German → all UI strings update
- [ ] Refresh after switch → language persists
- [ ] Submit report in English → tracking page respects locale

---

6. Production Hardening

Supabase

- [ ] *Settings → Auth → Email* — disable email provider (app uses custom HMAC cookie auth, not Supabase Auth)
- [ ] *Settings → API → Allowed origins* — restrict to `https://greececlean.gr`
- [ ] Enable **Point-in-Time Recovery** on Pro plan
- [ ] Set storage bucket file size limit

Vercel

- [ ] Enable **Bot Protection** (*Settings → Security*)
- [ ] Set `/api/report` function max duration to **30 seconds** (*Settings → Functions*)
- [ ] Confirm `SUPABASE_SERVICE_ROLE_KEY` is scoped as **Server** not **Browser**

---

7. Post-Launch Monitoring

First 48 hours

- [ ] Watch Vercel *Functions* logs for unhandled errors from `/api/report`
- [ ] Check Supabase *Logs → API* for 500s or unexpected RLS violations
- [ ] Check Resend *Logs* for bounced emails after first real forward

Ongoing

- [ ] Query `email_logs WHERE status = 'failed'` to catch stale municipality addresses
- [ ] Monitor Supabase Storage usage (each report ≤ 500 KB WebP)
- [ ] `ADMIN_COOKIE_SECRET` rotation procedure: update in Vercel → redeploy → all sessions invalidate

Schema verification SQL (run after migration to confirm setup):

```sql
-- RLS enabled
SELECT tablename, rowsecurity FROM pg_tables
WHERE schemaname = 'public'
  AND tablename IN ('reports', 'municipalities', 'email_logs');

-- Policies
SELECT tablename, policyname, cmd FROM pg_policies
WHERE schemaname = 'public' ORDER BY tablename, cmd;

-- Storage bucket
SELECT name, public FROM storage.buckets WHERE name = 'reports';

-- updated_at trigger
SELECT trigger_name, event_object_table FROM information_schema.triggers
WHERE trigger_name = 'reports_updated_at';

-- Municipality count
SELECT COUNT(*), COUNT(email_official) AS with_email FROM municipalities;
```

Security & Quality Audit Report

Audit performed: May 2026 — pre-handover

Summary of Findings

Priority	Issue	File	Status
P0 Critical	Admin routes completely unprotected	`proxy.ts` / `middleware.ts`	**Fixed**
P1 Bug	React Fragment missing `key` prop	`AdminReportList.tsx`	**Fixed**
P1 Bug	No try/catch on `req.formData()`	`app/api/report/route.ts`	**Fixed**
P1 Bug	No try/catch on `req.json()`	`app/api/admin/report/[id]/route.ts`	**Fixed**
P1 Bug	Admin actions swallow all errors silently	`AdminReportList.tsx`	**Fixed**
P1 Bug	Landing page data fetchers can crash the page	`app/(public)/page.tsx`	**Fixed**
P1 Bug	GPS handler ignores missing `navigator.geolocation`	`LocationPicker.tsx`	**Fixed**
P2 Security	XSS via unescaped HTML in Leaflet popups	`MapClient.tsx`	**Fixed**
P3 UX	Copy button shows wrong label in both states	`ReportForm.tsx`	**Fixed**
P4 DX	Generic error replaces descriptive compression error	`app/api/report/route.ts`	**Fixed**

Detailed Findings & Fixes

P0 — Critical: Admin Routes Unprotected

Root cause: `proxy.ts` existed with correct authentication logic but in the wrong filename. Next.js middleware **must** be in a file named `middleware.ts` at the project root, exporting `default function middleware()`. The `proxy.ts` file was never executed by the framework.

Impact: Any unauthenticated user could access `/admin/dashboard`, `/admin/municipalities`, and all admin API mutation routes.

Fix: Created `middleware.ts` at the project root with `export default function middleware()` and `export const config`. The old `proxy.ts` is now dead code and can be deleted.

P1 — Bug: React Fragment Key in AdminReportList

Root cause: The `reports.map()` loop returned `<> … </>` (shorthand fragment) wrapping each pair of rows. The `key` prop was on the inner `<tr>` instead of the fragment wrapper. React requires keys on the outermost element returned from `.map()`.

Impact: React key warnings in the console; potential render instability when the inline edit row was toggled, causing the wrong row to expand.

Fix:

```
// Before
{reports.map((r) => (
  <>
    <tr key={r.id}> …

// After
{reports.map((r) => (
  <Fragment key={r.id}>
    <tr> …
```

P1 — Bug: Unhandled Body Parsing Errors in API Routes

Root cause: `req.formData()` and `req.json()` were called without try/catch. A malformed multipart request or invalid JSON body would throw an unhandled exception, producing a generic 500 error.

Fix: Both calls now wrapped in try/catch returning 400 with a descriptive message.

P1 — Bug: Admin Actions Swallow Errors Silently

Root cause: `runAction()` in `AdminReportList` called `router.refresh()` unconditionally, even when the API returned an error response.

Impact: A failed approve/reject/delete action would appear to succeed — the table would refresh with no visible change and no user feedback.

Fix: `runAction()` now checks `res.ok`, parses the error message, and shows an `alert()` to the admin. Network failures are also caught and reported.

P1 — Bug: Landing Page Can Crash on DB Error

Root cause: `getStats()` and `getLeaderboard()` called Supabase directly with no error handling. A database timeout or network error would throw, causing Next.js to render the error page instead of the landing page.

Fix: Both functions wrapped in try/catch, returning `{ total: 0, resolved: 0, municipalities: 0 }` and `{ champions: [], needsWork: [] }` respectively on failure.

P2 — Security: XSS in Leaflet Map Popups

Root cause: `MapClient.tsx` built Leaflet popup HTML via string interpolation:

```
.bindPopup(`<p>${municipalityName}</p><p>${categoryLabel}</p>
            <a href="/r/${r.public_token}"> … </a>`)
```

If any of these strings contained `<script>`, `<img onerror= … >`, or other HTML, it would execute in the user's browser when the popup opens.

Fix: Added `escHtml()` function and applied it to all four interpolated values.

```
function escHtml(s: string): string {
  return s.replace(/&/g,'&amp;').replace(/</g,'&lt;')
          .replace(/>/g,'&gt;').replace(/"/g,'&quot;')
          .replace(/'/g,'&#x27;')
}
```

P3 — UX/i18n Bug: Copy Button Wrong Labels

Root cause: The success step in `ReportForm` had its own copy button that used `t.successLinkLabel` ("Tracking link") for **both** copied and not-copied states. The dedicated `CopyButton` component correctly used `t.copy.copy` / `t.copy.copied`.

Fix: `ReportForm` now accepts a `copyTranslations: Dictionary['copy']` prop. The report page passes `{ copy: c }` from `getDictionary()`. The button now shows the correct "📋 Copy Link" / "✓ Copied!" labels in all three languages.

What Was Already Correct

- **RLS policies** — anon users cannot UPDATE or DELETE reports ✓
- **Service role key** — no `NEXT_PUBLIC_` prefix, server-only ✓
- **Honeypot** — returns convincing fake success to confuse bots ✓
- **All three i18n dictionaries** — complete, no missing keys ✓
- **Image compression** — three-tier sharp pipeline is correct ✓
- **HMAC session logic** — correctly implemented ✓

Project Retrospective

A learning document for future projects of this type.

What the Git Log Shows

The project was built in **24 commits** across five distinct phases:

Phase	Commits	Description
1	1–3	Core app, dependency fix, CVE patch
2	4–12	Seed data, admin auth, municipality logic
3	13–16	Leaderboard, tracking page, i18n, email
4	17–18	EXIF GPS extraction, camera picker
5	19–24	**4 consecutive commits fixing iPhone photo picker**

Critical observation: Admin authentication was **commit #5** — added as an afterthought after the core app was built. When added, it produced `proxy.ts` with a named export in the wrong file. The admin area was silently open to the world for the rest of the build.

The iOS story is equally telling: **4 commits to fix a single UI element**. The file picker on iPhone Firefox/Chrome/Safari required specific `<label htmlFor>` nesting that was discovered through trial-and-error. Had "works on iPhone Safari" been a stated requirement before line one, the solution would have been researched once.

Root Cause Taxonomy

Every bug and rework in this project falls into one of four categories:

A. Framework convention gaps *(most dangerous)*

The code is logically correct but in the wrong file, with the wrong export name, or missing a required annotation.

Issue	Root cause
`proxy.ts` instead of `middleware.ts`	Prompt said "add admin auth", not "create `middleware.ts` at root with `export default function middleware()`"
Missing `force-dynamic` on two pages — two fix commits	Prompt said "make the dashboard live", not "add `export const dynamic = 'force-dynamic'` to every DB-reading page"
`sharp` almost used on Edge Runtime	Runtime not specified upfront

B. Platform behavior not researched upfront *(the rework factory)*

Issue	Root cause
4 iOS fix commits for photo picker	"Add camera and file picker" — no mention of iPhone Safari/Firefox/Chrome quirks
`react-leaflet` /React 19 compat issue needing `.npmrc`	Stack chosen without checking peer dependency matrix
EXIF outside Greece — 2 fix commits	Edge case (photo taken abroad) not specified

C. Missing defensive requirements *(audit findings)*

Every issue the audit found is an *absence* — something the prompt never asked for. An AI optimises for the happy path unless told otherwise.

Issue	Root cause
No try/catch on body parsing	Prompt said "handle submissions", not "handle malformed submissions"
XSS in Leaflet popup	Prompt said "show reports on map", not "escape all user data in HTML contexts"
Landing page crashes on DB error	Prompt said "show live stats", not "degrade gracefully if DB is unreachable"

D. Incremental feature drift *(coherence erosion)*

When features are added commit by commit without a complete spec, components diverge.

Issue	Root cause
Copy button showed "Tracking link" in both states	`CopyButton` built correctly later, but success step had its own independent copy button
`lib/notifications.ts` became dead legacy	Email rebuilt in `emailTemplates.ts` without removing the old file
`MunicipalityEmailList` still has Fragment key bug	`AdminReportList` was fixed in audit; the parallel component built in the same session was missed

---

The 15 Clarifying Questions

These are the specific questions that — if answered before the first token was generated — would have eliminated every rework in this project.

Framework & Runtime

1. Next.js App Router or Pages Router?
2. Any reason to use Edge Runtime? If not, state all API routes use Node.js runtime.
3. What is the deployment target?

Authentication

4. Single shared password, or multiple named users?
5. Which routes are protected? (Exact list — vague answers produce the `proxy.ts` mistake.)
6. What is the session duration and invalidation mechanism?

Database

7. What is the complete report status lifecycle including valid transitions?
8. What are the exact RLS rules? (Explicit `SELECT/INSERT/UPDATE/DELETE` per role.)
9. Hard delete or soft delete?

Mobile & Browser Support

10. Which mobile browsers must work for the report form? (iPhone Safari, Chrome for iOS, Firefox for iOS, Android Chrome — each has different file input behavior.)
11. Is live camera capture via `getUserMedia` required, or is a file picker with `capture` attribute acceptable?

Internationalisation

12. Is the admin area translated or single-language?

13. Are email templates language-aware or always sent in a fixed language?

Resilience

14. What should the app do when Supabase is unavailable?
15. What should happen if a critical third-party service fails (Nominatim, Resend, Storage)?

The Ideal Initial Prompt

This is the prompt that would have produced the final audited state in a single pass, with no rework commits.

> ***Build GreeceClean*** *— a Next.js 16 App Router web application for citizens to report environmental violations in Greece.*
>
> ***TECHNICAL CONSTRAINTS — non-negotiable***
>
> - *Next.js 16, App Router, TypeScript strict mode, Tailwind CSS*
> - ***Node.js runtime for all API routes — never Edge Runtime*** *(sharp requires native binaries)*
> - `serverExternalPackages: ['sharp']` *in* `next.config.ts`
> - *All DB-reading pages must have* `export const dynamic = 'force-dynamic'`
> - *Every Leaflet component must use* `dynamic(() => import(...), { ssr: false })`
> - *Route protection via* `middleware.ts` ***at the project root*** *— not any other filename — exporting* `default function middleware()` *and* `export const config`
>
> ***AUTHENTICATION***
>
> - *Single shared password in* `ADMIN_PASSWORD`*. No user accounts in the database.*
> - *Session: HMAC-SHA256(* `ADMIN_PASSWORD` *,* `ADMIN_COOKIE_SECRET` *), cookie* `admin_session` *, HttpOnly, SameSite=Lax, 8-hour MaxAge.*
> - *Protected: all* `/admin/*` *and* `/api/admin/*` *except* `/admin/login` *,* `/api/admin/login` *,* `/api/admin/logout` *.*
>
> ***DATABASE — Supabase***
>
> *Three tables:* `reports` *,* `municipalities` *,* `email_logs` *. RLS explicit:*
>
> - `reports` *— anon: SELECT where* `is_approved=true` *, INSERT unrestricted. No anon UPDATE or DELETE.*
> - `municipalities` *— anon: SELECT only.*
> - `email_logs` *— no public policies.*

> ***PHOTO UPLOAD***
>
> - *Two options: `getUserMedia` live viewfinder (desktop + Android) AND plain `<input type="file" accept="image/*">` inside `<label>` with **NO** `capture` **attribute** (iOS compatibility — `capture="environment"` breaks file picker on iPhone Firefox/Chrome).*
> - *EXIF GPS via `exifr.gps()`. Pre-fill map if within Greek bounding box (34.8–41.8 lat, 19.3–29.7 lng). If GPS found but outside Greece, show amber warning "GPS found outside Greece — please select on the map." EXIF failure is non-fatal.*
> - *Server-side compression via sharp: WebP, three passes (1920px/q80 → 1200px/q65 → 900px/q50) until ≤ 500 KB. Pass through the specific error message if compression fails.*
>
> ***LOCATION***
>
> - *Leaflet map picker: click-to-place, draggable marker, GPS button (guard `if (!navigator.geolocation) return`), Nominatim search debounced 500ms with `countrycodes=gr`.*
> - *Server-side reverse geocoding via Nominatim zoom=10, fallback zoom=8. Auto-create municipality row if unknown name returned.*
>
> ***EMAIL***
>
> - *Provider: Resend. Lazy-init client.*
> - *Forward action: update `status='forwarded'` first, then attempt email. If email fails: log to `email_logs` with `status='failed'`, return HTTP 207 with `warning` field. Never block status update on email success.*
> - *Templates always in Greek. Escape all user content before inserting into HTML.*
>
> ***INTERNATIONALISATION***
>
> - *Three locales: `el` (default), `en`, `de`. Cookie-based, not HttpOnly.*
> - *Admin area: intentionally Greek-only — do not translate admin strings.*
>
> ***ERROR HANDLING — mandatory everywhere***
>
> - *Wrap `req.formData()` and `req.json()` in try/catch; return 400 on failure.*
> - *Wrap all page-level Supabase calls in try/catch; return safe defaults on failure — never let a DB error crash a page render.*
> - *Escape all user-controlled strings before inserting into any HTML context.*
> - *Guard `navigator.geolocation`, `navigator.mediaDevices`, `navigator.clipboard` before calling.*
>
> ***STUB MODE***

> *If `NEXT_PUBLIC_SUPABASE_URL` absent: use `lib/seed-data.ts` (17 sample reports) for all reads. Report submission returns fake token + `_stub: true`. Admin API routes return 503.*

The Three Rules

1. Conventions before logic. Framework rules — file names, export shapes, runtime declarations, caching annotations — must be in the prompt. The AI derives the logic; it cannot reliably derive undocumented naming conventions.

2. Edge cases are features. "Add GPS extraction" and "add GPS extraction that handles photos taken outside Greece on iPhone Firefox" are different prompts producing different code. Every edge case left out of the prompt becomes a fix commit.

3. Robustness is a separate requirement. A feature prompt describes the happy path. A separate paragraph stating "*all async operations must have try/catch, all user data must be escaped, all optional browser APIs must be guarded*" produces defensive code. Without that paragraph, the AI optimises for correctness-when-working, not resilience-when-failing.

Pre-Kickoff Stakeholder Interview Questions

Role: Technical Lead / Project Manager preparing for project kickoff.
Every question is grounded in a specific issue encountered during the GreeceClean build.

Meeting 1 — CEO / Project Sponsor

On vision and success

1. In 12 months, what does success look like in concrete numbers? Are we measuring reports submitted, municipalities responding, cases resolved, or returning citizens? The database schema, the analytics layer, and the landing page all change depending on this answer.
2. Who is the primary user you are optimising for — the citizen, the municipality, or the admin? When these three users want conflicting things, whose experience wins?
3. Is this a standalone product, or does it need to integrate with existing government systems? A platform that will connect to a municipal ERP is architected very differently from one that operates independently.

On scope and timeline

4. Do you have a hard launch date tied to an external event? A hard deadline changes every prioritisation decision. I need to know on day one, not week eight.
5. What is the minimum feature set that must be live at launch for the project to be considered a success? I need "must have at launch" clearly separated from "nice to have eventually."
6. After launch, who owns the product? A dedicated internal person, or a maintenance contract with us? The answer changes how much we document and how opinionated we make the codebase.

On risk

7. What is the highest-risk failure scenario you are imagining? A report goes viral with 10,000 hits in an hour? A municipality complains about spam? A data exposure in the press? Each failure mode requires different mitigation priority.
8. Is there a budget ceiling for infrastructure costs per month? "Unlimited" and "€50/month" produce completely different architecture decisions.

Meeting 2 — Marketing & Communications

On audience

1. Is this targeting Greek citizens primarily, or do you expect significant non-Greek tourist usage? This decides whether English and German are first-class citizens or optional extras — and that changes the entire i18n architecture.
2. What age range is the primary target? A 60-year-old on a basic Android phone and a 25-year-old on iPhone have different expectations from a camera and map. Which user wins when there's a tradeoff?
3. Do you expect users to share reports on social media? If yes, every tracking page needs Open Graph metadata, preview images, and canonical URLs from day one. This is not retrofittable cheaply.

On brand

4. Do you have a finished brand identity — logo, hex colours, typography — or will that be delivered during development? If the design system is not locked before we build components, we will rebuild UI twice.
5. What is the final domain name? I need this before the first commit. The domain is baked into database rows, email templates, and environment variables. Changing it later means touching stored data.

On growth mechanics

6. What is the primary acquisition channel — social, word of mouth, municipality partnerships, press? This tells me whether the WhatsApp share button is a core feature or a nice-to-have.
7. Do you plan paid campaigns requiring UTM tracking or pixel integration? Injecting analytics into a Next.js App Router project retroactively requires careful setup to avoid breaking Core Web Vitals.
8. Do municipalities need co-branded versions — e.g. a dedicated subdomain per municipality? Multi-tenancy is an architectural decision, not a feature switch.

Meeting 3 — UX / Design

On flows

1. Walk me through the report submission flow step by step. How many steps? Can the user go backwards? What happens if they close the browser mid-way? Each answer is a feature.

2. What does the citizen expect after submitting? Do they come back daily? Do they want notifications? Is the tracking page a "peace of mind" feature or an active engagement surface? This determines whether we need push notifications.
3. What is the empty state on the map when there are zero reports? What does the landing page show when all stats are zero? These must be designed — a map with zero markers looks broken and damages trust at launch.

On design system

4. Are you delivering a complete Figma component library, or am I building the design system from the brief? Who has final sign-off on component decisions?
5. What are the exact brand colours in hex? Primary, secondary, error, success, warning. "Blue and green" is not enough to start building.
6. Are there accessibility requirements? WCAG 2.1 AA? Greek public sector digital accessibility compliance?

On mobile

7. Is the report form the primary mobile use case and the admin primarily desktop? Do administrators also work from phones? This affects touch target sizes and table layouts.
8. Do you expect citizens to be outdoors in sunlight with one hand occupied when submitting? If yes, contrast ratios and button sizes need to exceed standard guidelines. "Mobile-responsive" and "field-usable" are different requirements.

Meeting 4 — Frontend Development

On framework and conventions

1. Are we using Next.js App Router or Pages Router? This is not a discussion — I need a definitive answer before the first file is created.
2. What is the convention for the server/client component boundary? Proposal: server components for all pages and data fetching, client components only when we need browser APIs. Can we document this as a team rule?
3. What is the exact file name and export shape for our Next.js middleware? Answer: `middleware.ts` at the project root, `export default function middleware()`, `export const config`. Can we make this a code review checklist item?
4. For any component using Leaflet or any browser-only library: what is the rule? Answer: `dynamic(() ⇒ import( … ), { ssr: false })` always. Can we add an ESLint note for this?

On browser and device support

5. Which mobile browsers are explicitly in scope? I need a written list: iPhone Safari, Chrome for iOS, Firefox for iOS, Samsung Internet, Android Chrome. Any browser not on this list is not tested and not supported.
6. Specifically for the photo picker: have we validated that `<input type="file">` inside `<label htmlFor>` works on our target devices? Or do we need to prototype this before committing to the approach?
7. Are there known low-end device targets? €80 Android phones with 2 GB RAM change our JavaScript bundle strategy.

On state and data

8. Where is client state managed? For this scale, local `useState` should be sufficient — no Redux or Zustand. Agreed?
9. How do we handle failed admin actions? Show alert, silent refresh, or inline error message? Decide before building, not after.

Meeting 5 — Backend Development

On authentication

1. Single shared admin password or individual user accounts with roles? Single password is 2 hours of work. Individual accounts is 2 weeks. I need this on day one — everything changes.
2. If single password: what is the session invalidation procedure if the password is compromised?
3. Do municipalities ever log in directly, or do they only receive emails? A municipality portal is a completely different product. If it is in scope for v2, the data model must support it now.

On the database

4. What is the complete report status state machine — every state, every valid transition, every terminal state? I need this drawn before the schema is written. An enum that needs a new value in production requires a live migration.
5. Hard delete or soft delete for reports? If a report is deleted, does the image disappear? What is the cascade behavior for all foreign keys?
6. Should municipality names be normalised? The current approach auto-creates new rows if Nominatim returns unknown names, which can produce duplicates ("Αθήνα" vs "Δήμος Αθηναίων"). Is that acceptable, or do we need a merge mechanism?

On integrations

7. Nominatim for geocoding or Google Maps API? Nominatim is free but rate-limited and has data gaps. Google Maps is paid but reliable. Budget decision.
8. If a municipality email bounces, what is the recovery process? Does the admin manually resend? Is there a notification to the GreeceClean team?
9. Do municipalities need a callback API to update report status, or is all management done by the admin? The answer determines whether this is a manual workflow or an automated platform.

On security

10. What are the exact RLS rules per role per operation? I need this stated explicitly before writing a single policy.
11. Is a honeypot sufficient bot protection, or do we need Turnstile CAPTCHA? A determined actor can bypass a honeypot. What is our threat model?

Meeting 6 — DevOps / Infrastructure

On hosting

1. Is Vercel the confirmed platform? Which plan? The Hobby plan has function execution limits that matter for `sharp` image compression.
2. Is Supabase confirmed? Which region? Are there data sovereignty requirements (data must remain in the EU)?
3. What is expected peak traffic? "1,000 users on day one from a press announcement" requires a different scaling plan than slow organic growth.

On environments

4. How many environments? Development, staging, production? Are staging and production on the same Supabase project or separate ones? Running staging against production data is dangerous.
5. Who manages environment variable rotation? If `ADMIN_COOKIE_SECRET` is rotated, every admin session is invalidated. This needs a documented runbook and a responsible person.
6. How is `ADMIN_PASSWORD` distributed to the team? I will not send plaintext production credentials over unencrypted channels.

On observability

7. What is the alerting strategy for production errors? Vercel logs are not alerted. Do we integrate with Sentry or Datadog?

8. Who is on-call when the platform has an incident on a Saturday night? What is the escalation path?
9. Do we need database backup beyond Supabase defaults? The free plan has point-in-time recovery disabled.

On CI/CD

10. Is there a required code review process before production deployment? Or can a single developer push directly? I need this defined before the first pull request.

Meeting 7 — Legal & Compliance

On GDPR

1. Does submitting a report constitute processing of personal data? The photo may contain faces or licence plates; GPS coordinates identify a physical location.
2. What is the legal basis for processing this data? Legitimate interest? Public interest task? Explicit consent? This determines whether we need a consent checkbox on the form.
3. What is the data retention policy? How long are resolved reports kept? Rejected reports? Do we need an automated deletion job?
4. Do users have the right to delete their own reports? There is no user account — only a token. Does token possession constitute "identity" sufficient to exercise deletion rights?

On municipalities

5. Do we have data processing agreements (DPAs) with every municipality that receives report emails? They are receiving citizen data without those citizens explicitly consenting to transfer to that specific municipality.
6. Who is legally responsible for report content? If a citizen submits a false claim about a private property and a municipality acts on it, what is GreeceClean's liability?

On launch

7. Do we need Terms of Service and a Privacy Policy linked from the report form before launch? In the EU, collecting this data without clearly accessible legal documents is a compliance breach.

Meeting 8 — Municipal Affairs

On workflow

1. What email address does each municipality want to receive reports at? Generic inbox, specific department, or named individual? How do we learn when it changes?
2. Who reads the notification email? A secretary who forwards internally? A specific environmental officer? The actual reader changes the required content and tone.
3. When a municipality receives the email, what do they do with it? Log it in an internal case system? Is the email itself the workflow trigger? This determines whether we need status-update callbacks.

On technical constraints

4. What email clients do municipality staff use? Outlook 2016 on Windows is common in Greek public sector and has limited HTML rendering. Our template must work in it.
5. Do municipality email servers apply aggressive spam filtering? A `@greececlean.gr` sender not previously seen will be filtered without proper SPF/DKIM/DMARC records and possibly manual whitelisting.

On accountability

6. Is a municipality obligated to act on a received report, or is participation voluntary? This determines whether "no response" requires an escalation path.
7. Should a municipality be able to update a report's status directly — e.g. "crew dispatched" or "resolved"? A municipality-facing portal is a significant scope increase; if it is needed in v2, the data model must support it now.

Meeting 9 — QA / Testing

On coverage

1. What is the definition of "done" for a feature? Passes unit tests? Passes E2E tests? Passes manual test on a physical device? All of the above? This must be written into acceptance criteria before development starts.
2. Do we have automated E2E tests for the report submission flow? This is the single most critical user path. A regression here that reaches production silently is an incident.
3. What is the test strategy for the admin dashboard? Complex state (inline edit, multi-step actions, confirm dialogs). Unit tests, integration tests, or manual only?

On devices

4. What physical devices do we have in the QA lab? I need at minimum: one iPhone on Safari, one iPhone on Chrome for iOS, one Android on Chrome. Simulators do not substitute for real devices when testing camera and file picker APIs.
5. Who is responsible for regression testing after each deployment?

On acceptance

6. What are the performance budgets? Acceptable page load time on 4G for the report form? For the public map? Adjectives like "fast" are not measurable.
7. Is there a Lighthouse score requirement? Without a number, "good enough" becomes the standard.
8. Who has final sign-off before production — QA, Product, or Client? And what is the escalation path when a critical bug is found three days before launch?

The Dependency Map

These meetings are not independent. Several answers unlock or block other meetings:

```
CEO: "Hard launch date — press event in 8 weeks"
  ↳ DevOps: Skip multi-environment setup. One environment, deploy fast.
  ↳ Legal: Fast-track Privacy Policy. Minimum viable compliance.

Legal: "Consent checkbox required before form submission"
  ↳ UX: Add consent step to the report form wireframes.
  ↳ Backend: Add consented_at column to reports table.
  ↳ Frontend: Disable submit until consent is checked.

Municipal Affairs: "Staff use Outlook 2016"
  ↳ Backend: Email template must be HTML table layout, no flexbox, no CSS grid.
  ↳ QA: Add Outlook 2016 to the email rendering test matrix.

CEO: "Municipality direct login is v2 scope"
  ↳ Backend: Add municipality_user table now — do not normalise in a way that blocks
this.
  ↳ Auth: Design session system to support two user types from day one.

Frontend: "Must support iPhone Firefox and Chrome for iOS"
  ↳ UX: Camera button must be hidden on those browsers (no getUserMedia support).
  ↳ QA: Add iPhone Firefox and Chrome for iOS to the mandatory test matrix.
```

The One-Page Brief Template

After all nine meetings, consolidate every answer into this brief before writing the first prompt:

```
PROJECT BRIEF
──────────────────────────────────────────────────
Product name:       [confirmed, final]
Domain:             [confirmed before any code]
Launch date:        [hard / soft — if hard, specific date]
Primary user:       [citizen / admin / municipality — ranked]
Languages:          [list — admin included or excluded]
Device support:     [exact browser / OS list]

TECHNICAL STACK
──────────────────────────────────────────────────
Framework:          Next.js [version] App Router, Node.js runtime, TypeScript strict
Database:           Supabase, region [x], tables [list with RLS rules per role]
Storage:            Supabase Storage, bucket [name], visibility [public/private]
Email:              [provider], always in [language], sender domain [x]
Deployment:         Vercel [plan], [number of environments]

AUTHENTICATION
──────────────────────────────────────────────────
Admin:              Single password / multi-user accounts
Session:            [duration], [invalidation mechanism]
Protected routes:   [exact list]

DATA MODEL
──────────────────────────────────────────────────
Status state machine: [complete diagram]
Delete behavior:      hard / soft
Retention policy:     [from legal]
GDPR legal basis:     [from legal]

QUALITY GATES
──────────────────────────────────────────────────
Definition of done:   [explicit — written, not verbal]
Mobile test devices:  [exact list]
Performance budget:   [Lighthouse score / load time numbers]
Sign-off authority:   [named person]
```

That document is the initial prompt. Give it to the AI — and you build once.

End of GreeceClean Complete Project Handover Documentation

Prepared May 2026 · Version 1.0 · Confidential